

对抗攻击与对抗训练实验报告

以 DeepFool 为核心：模型训练 · 攻击 · 防御

王军 梁利国 汤付伟

Co-authored with Claude Opus 4.8

June 15, 2026

目录

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果
- 5 对抗训练（防御）
- 6 讨论与总结

本章内容

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果
- 5 对抗训练（防御）
- 6 讨论与总结

什么是对抗样本？

核心现象

给一张能被模型正确分类的图像，叠加一个**人眼几乎察觉不到的**微小扰动，模型就会以高置信度把它分错。这个被扰动后的图像称为**对抗样本**。

- 扰动 $\mathbf{r}$ 很小： $\|\mathbf{r}\|$ 接近 0，视觉上与原图无差别。
- 但模型输出 $\hat{k}(\mathbf{x} + \mathbf{r}) \neq \hat{k}(\mathbf{x})$ 。
- 说明深度网络的决策边界在高维空间里非常“贴近”正常样本。

研究意义：评估模型鲁棒性、理解决策边界、改进训练（对抗训练）。

本次实验的三条主线

① 训练模型

在 CIFAR-10 上微调 ResNet-18，得到一个准确的**基线模型**，作为被攻击对象。

② DeepFool 攻击

(核心) 用最小扰动把基线模型骗倒，并与 FGSM / APGD 对比。

③ 对抗训练

用 PGD 对抗训练得到**鲁棒模型**，诚实评估防御是否真有效。

训练基线模型

DeepFool 攻击

对抗训练 + 评估

攻击是“矛”，对抗训练是“盾”；先有准确的模型，才谈得上攻击与防御。

本章内容

- 1 背景与实验目标
- 2 模型训练**
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果
- 5 对抗训练（防御）
- 6 讨论与总结

CIFAR-10: 10 类自然图像, 原生 32×32 。

- 训练 / 验证 / 测试 = **45000 / 5000 / 10000**
- 数据增广: `RandomCrop(32, padding=4)` + `RandomHorizontalFlip`
- 归一化: 均值 $[0.485, 0.456, 0.406]$, 标准差 $[0.229, 0.224, 0.225]$
- 攻击阶段图像保持在 $[0, 1]$ 像素空间, **归一化包进模型内部** (避免攻击库的 `clamp(0, 1)` 破坏对抗样本)。

为什么把归一化放进模型

攻击需要在合法像素区间 $[0, 1]$ 内裁剪。若先归一化再攻击, `clamp(0, 1)` 会把归一化后的负值截断、毁掉图像。统一约定: **数据 $[0, 1]$, 归一化作为模型第一层。**

模型结构：CIFAR 版 ResNet-18

直接用 ImageNet 版 ResNet-18 会把 32×32 一上来就压到 8×8 ，丢失太多空间信息。需做两处**适配小图**的改造：

- conv1: 7×7 stride 2 $\rightarrow 3 \times 3$ **stride 1, padding 1**
- maxpool: $\rightarrow$ **Identity (去掉)**
- fc: 输出维度改为 **10 类**

- 参数量 ≈ 11.2 M，从零开始训练 (weights=None)。
- 固定随机种子 seed=42，保证可复现。

训练配置

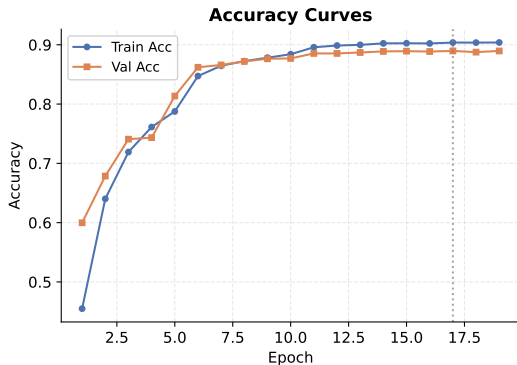
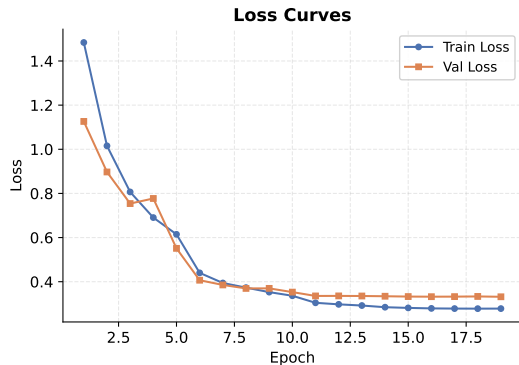
- 损失：交叉熵
- 优化器：Adam, $lr = 10^{-3}$, weight decay = 10^{-4}
- 学习率调度：StepLR (每 5 轮 $\times 0.1$)
- 早停：验证准确率连续 **5** 轮不提升即停
- 最多 30 轮 (早停自动截断)

结果 (实际训练 **19** 轮, 早停于第 17 轮取最佳)

指标	数值
最佳验证准确率	88.96%
测试 Top-1 准确率	88.53%
测试 Macro-F1	88.50%

该模型作为后续所有攻击 / 防御实验的**基线 (普通模型)**, 权重存于 resnet18_cifar10_best.pth。

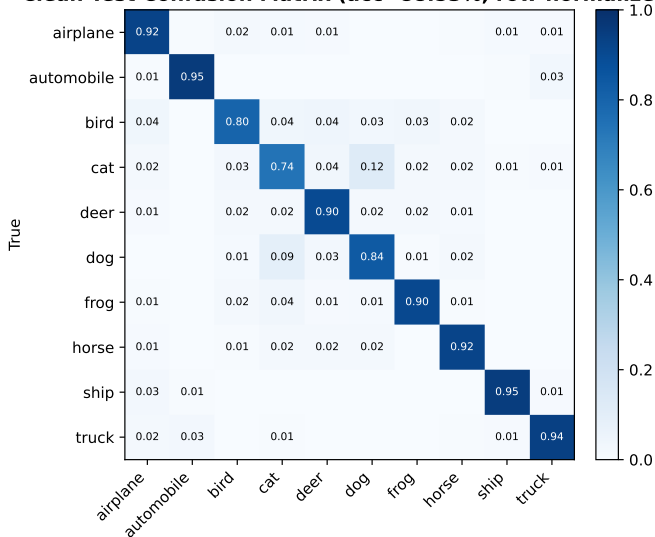
训练曲线 (Loss / Accuracy)



约第 6 轮 (StepLR 首次衰减) 后曲线明显跃升并迅速收敛；验证准确率在第 17 轮达到峰值 **88.96%**，此后连续 5 轮无提升触发早停 (虚线处为最佳轮)。

基线模型：干净测试集混淆矩阵

Clean Test Confusion Matrix (acc=88.53%, row-normalized)



行归一化（每行 = 该真实类别被预测到各类的比例）。

- 对角线普遍 **0.8–0.95**，整体准确率 **88.53%**。
- 最易混的是 **cat ↔ dog**（cat 仅 0.74，最低）。
- automobile/ship/truck 最稳（ ≥ 0.93 ）。
- 误分主要发生在**语义相近**的类别之间，符合直觉。

本章内容

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）**
- 4 攻击实验与结果
- 5 对抗训练（防御）
- 6 讨论与总结

对抗攻击的形式化定义

对于分类器 f 与样本 $\mathbf{x}$ ，记其预测标签

$$\hat{k}(\mathbf{x}) = \arg \max_k f_k(\mathbf{x}).$$

寻找“最小”对抗扰动可写成优化问题：

$$\Delta(\mathbf{x}; f) := \min_{\mathbf{r}} \|\mathbf{r}\|_2 \quad \text{s.t.} \quad \hat{k}(\mathbf{x} + \mathbf{r}) \neq \hat{k}(\mathbf{x}).$$

- $\Delta(\mathbf{x}; f)$ 度量了样本 $\mathbf{x}$ 到**决策边界**的距离。
- 这个距离越小，说明模型在该点越“脆弱”。
- **DeepFool 的目标：** 高效、准确地求解这个最小扰动。

回顾 FGSM 及其局限

最经典的攻击 FGSM (Fast Gradient Sign Method) 只走一步：

$$\mathbf{r} = \varepsilon \cdot \text{sign}(\nabla_{\mathbf{x}} J(\theta, \mathbf{x}, y)).$$

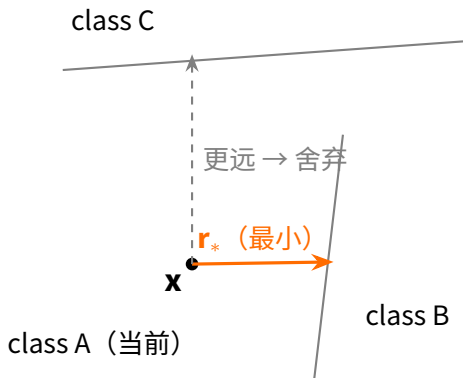
- **优点：** 单步、极快，直觉清晰（沿损失上升最快方向推一步）。
- **局限一：** 取 sign 把每个像素都顶到 $\pm\varepsilon$ ，扰动**粗糙且偏大**。
- **局限二：** 不是为“最小扰动”设计的， ε 需要**手动调**。

由此引出 DeepFool

能不能不手调参数，自动找到**刚好越过决策边界**的最小扰动？

几何直觉：决策边界就是“超平面”

- 在样本附近，把分类器**局部线性化**，决策边界近似为一个个**超平面**。
- 让样本改变类别，最省力的做法是：沿**垂直方向**推到**最近**的那个边界外。
- 这正是“点到平面的最短距离”——线性代数里的经典问题。



在所有相邻类别的边界中，DeepFool 选择**最近**的那条，做垂直投影。

二分类：线性分类器（最简单情形）

设二分类器 $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ ，预测 $\hat{k}(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$ 。

样本 $\mathbf{x}$ 到超平面 $\{\mathbf{x} : f(\mathbf{x}) = 0\}$ 的最小扰动，就是**正交投影**：

$$\mathbf{r}_*(\mathbf{x}) = -\frac{f(\mathbf{x})}{\|\mathbf{w}\|_2^2} \mathbf{w}$$

- 方向：沿法向量 $\mathbf{w}$ （边界的垂直方向）。
- 大小： $\|\mathbf{r}_*\|_2 = \frac{|f(\mathbf{x})|}{\|\mathbf{w}\|_2}$ ，正是**点到平面的距离**。
- 这是闭式解，一步到位。

二分类：一般（非线性）分类器

真实网络是非线性的，没有全局超平面。DeepFool 的做法：**反复线性化 + 迭代**。

在当前迭代点 $\mathbf{x}_i$ 把 f 一阶展开：

$$f(\mathbf{x}_i + \mathbf{r}) \approx f(\mathbf{x}_i) + \nabla f(\mathbf{x}_i)^\top \mathbf{r}.$$

令其等于 0，得到这一步的最小扰动：

$$\mathbf{r}_i = - \frac{f(\mathbf{x}_i)}{\|\nabla f(\mathbf{x}_i)\|_2^2} \nabla f(\mathbf{x}_i), \quad \mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{r}_i.$$

- 每走一步都重新计算梯度（重新线性化），逼近真实弯曲的边界。
- 直到 $\hat{k}(\mathbf{x}_i)$ 改变为止，通常只需 **2-4 步**。

多分类：在所有边界中选最近的一条

对 c 类分类器 $f: \mathbb{R}^n \rightarrow \mathbb{R}^c$ ，记原始标签 $\hat{k}(\mathbf{x}_0)$ 。在迭代点 $\mathbf{x}_i$ ，对每个其他类别 $k \neq \hat{k}(\mathbf{x}_0)$ 定义：

$$\mathbf{w}'_k = \nabla f_k(\mathbf{x}_i) - \nabla f_{\hat{k}(\mathbf{x}_0)}(\mathbf{x}_i), \quad f'_k = f_k(\mathbf{x}_i) - f_{\hat{k}(\mathbf{x}_0)}(\mathbf{x}_i).$$

- $\mathbf{w}'_k$ ：类别 k 与当前类之间“等分边界”的法向。
- f'_k ：当前点相对该边界的“带符号距离量”。

选出**最近**的边界：

$$\hat{l} = \arg \min_{k \neq \hat{k}(\mathbf{x}_0)} \frac{|f'_k|}{\|\mathbf{w}'_k\|_2}.$$

多分类：最小扰动公式

朝最近边界 $\hat{l}$ 做正交投影，得到本步扰动：

$$\mathbf{r}_i = \frac{|f'_{\hat{l}}|}{\|\mathbf{w}'_{\hat{l}}\|_2^2} \mathbf{w}'_{\hat{l}}$$

然后更新 $\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{r}_i$ ，重复直到类别改变。

最终累计扰动并做**过冲**（overshoot），确保确实越过边界：

$$\hat{\mathbf{r}} = (1 + \eta) \sum_i \mathbf{r}_i, \quad \eta = 0.02.$$

过冲系数 1.02 让扰动“多推一点点”，避免样本卡在边界上、因数值/量化又被推回正确类。

算法伪代码（多分类 DeepFool）

- 1 初始化 $\mathbf{x}_0 \leftarrow \mathbf{x}$, $i \leftarrow 0$
- 2 **while** $\hat{k}(\mathbf{x}_i) = \hat{k}(\mathbf{x}_0)$ **do**
- 3 **for** $k \neq \hat{k}(\mathbf{x}_0)$: $\mathbf{w}'_k \leftarrow \nabla f_k(\mathbf{x}_i) - \nabla f_{\hat{k}(\mathbf{x}_0)}(\mathbf{x}_i)$, $f'_k \leftarrow f_k(\mathbf{x}_i) - f_{\hat{k}(\mathbf{x}_0)}(\mathbf{x}_i)$
- 4 选最近边界 $\hat{l} \leftarrow \arg \min_{k \neq \hat{k}(\mathbf{x}_0)} \frac{|f'_k|}{\|\mathbf{w}'_k\|_2}$
- 5 $\mathbf{r}_i \leftarrow \frac{|f'_\hat{l}|}{\|\mathbf{w}'_{\hat{l}}\|_2} \mathbf{w}'_{\hat{l}}$, $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \mathbf{r}_i$, $i \leftarrow i + 1$
- 6 **end while**
- 7 **return** $\hat{\mathbf{r}} = (1 + \eta) \sum_i \mathbf{r}_i$

复杂度主要来自每步对 c 个类别求梯度；CIFAR-10 ($c = 10$) 规模下很快。本实验用 `torchattacks.DeepFool(steps=20, overshoot=0.02)`。

本章内容

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果**
- 5 对抗训练（防御）
- 6 讨论与总结

最小扰动攻击不看“成功率”，而看**平均/中位 L_2 扰动**——越小越说明模型脆弱。论文用**平均相对扰动**：

$$\hat{\rho}_{\text{adv}}(\mathbf{f}) = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \frac{\|\hat{\mathbf{r}}(\mathbf{x})\|_2}{\|\mathbf{x}\|_2}.$$

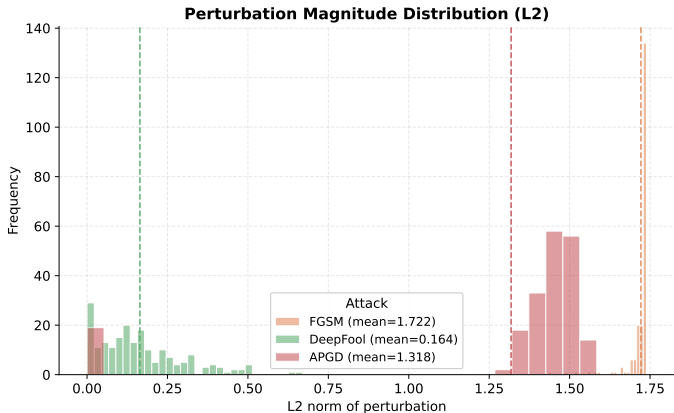
- **被攻击模型**：上一节训好的 ResNet-18（干净准确率 88.53%）。
- **攻击实现**：torchattacks，全部在 $[0, 1]$ 像素空间，取 200 个测试样本。
- **对照攻击**：FGSM、APGD（均 L_∞ ， $\varepsilon = 8/255$ ）。
- **成功率口径**：**条件攻击成功率** = 干净分对、却被攻击翻掉的比例。

实测结果：三种攻击对比（普通模型，白盒）

攻击	范数 / 预算	攻击成功率	Robust Acc	平均 L_2
FGSM	L_∞ , 8/255	97.91%	1.88%	1.7208
DeepFool	L_2 (最小扰动)	97.21%	2.50%	0.1604
APGD	L_∞ , 8/255	100.00%	0.00%	1.3048

- DeepFool 的平均 L_2 仅 **0.16**，约为 FGSM (1.72) 的 **1/11**、APGD (1.30) 的 1/8。
- 三者骗过率都很高，但 DeepFool 用**小一个量级**的扰动达成同样效果。
- 干净准确率均为 89.69%（200 样本子集口径）。

实测结果：扰动大小分布 (FGSM vs DeepFool)

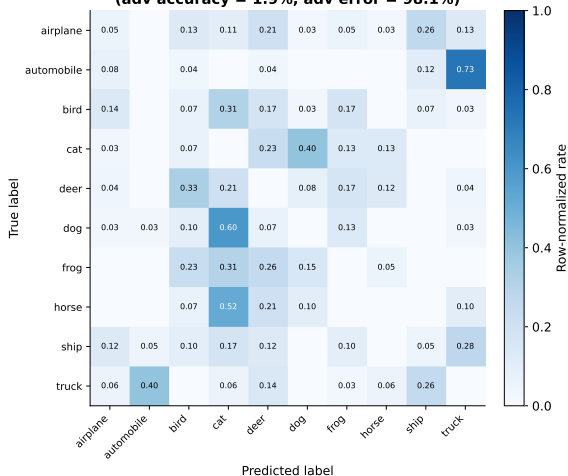


DeepFool 的 L_2 扰动集中在接近 0 的位置，整体分布远在 FGSM 左侧——这正是“最小扰动”路线的直接证据。

实测结果：混淆矩阵（对抗样本上）

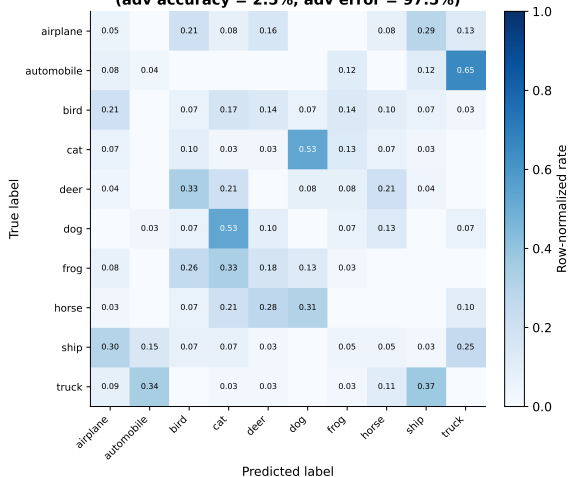
FGSM

FGSM: Confusion Matrix on Adversarial Samples
(adv accuracy = 1.9%, adv error = 98.1%)



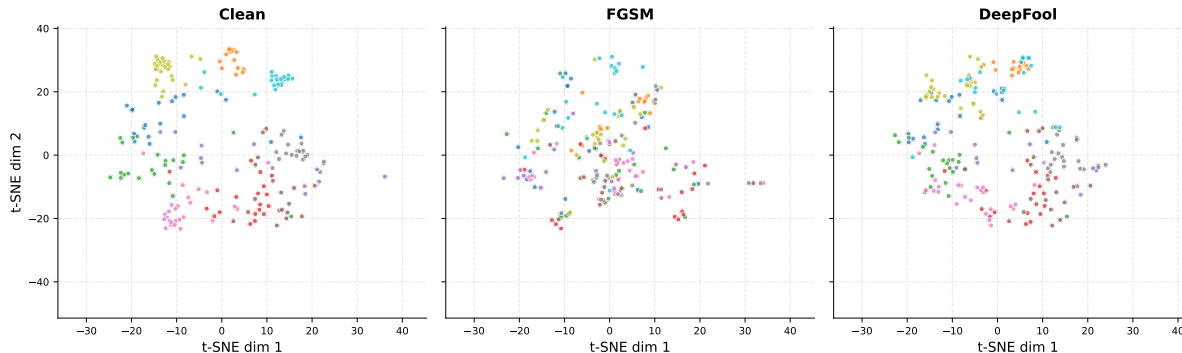
DeepFool

DeepFool: Confusion Matrix on Adversarial Samples
(adv accuracy = 2.5%, adv error = 97.5%)



实测结果：特征空间 t-SNE (Clean vs 对抗)

t-SNE by class: Clean vs FGSM vs DeepFool vs APGD (shared embedding)



共享嵌入下，干净样本按类别聚成清晰簇；对抗样本（FGSM/DeepFool/APGD）则被打散、跨越到其他类别的区域，直观展示决策被改写。

本章内容

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果
- 5 对抗训练（防御）**
- 6 讨论与总结

对抗训练：把“最坏样本”喂给模型

防御思路：训练时**先用 PGD 把每批干净图现场打成对抗样本**，再用它（标签仍为原始 y ）训练，逼模型“被攻击也分对”。形式化为**min-max（鞍点）问题**：

$$\min_{\theta} \mathbb{E}_{(\mathbf{x}, y)} \left[\max_{\|\delta\|_{\infty} \leq \epsilon} L(f_{\theta}(\mathbf{x} + \delta), y) \right].$$

- **内层 max**：在预算 ϵ 内找让损失最大的扰动（用 PGD 近似）。
- **外层 min**：更新参数 θ ，让“最坏情况”的损失也变小 $\Rightarrow$ 变鲁棒。
- 对抗样本**边训边生成**：模型每更新一次就变，需用当前模型重新打 PGD。

为什么用 PGD，而不是 DeepFool?

PGD (用于训练)

在**固定预算** ϵ **内最大化损失** $\rightarrow$ 给“最坏情况”样本，正好是内层 $\max L$ 要的。

像“在规则内全力打你”的陪练 $\rightarrow$ 越打越抗揍。

DeepFool (不用于训练)

求**最小扰动、刚越界就停** $\rightarrow$ 样本“贴在边界上”、很弱，且每个样本扰动大小不一、**没有统一预算**。

像“碰一下就收手”的陪练 $\rightarrow$ 练不出鲁棒性。

结论

PGD 同时用于训练与评估；**DeepFool** 是**评估**工具（测最小扰动大小、看到决策边界的距离）。

PGD 对抗训练配方（本实验）

- 训练集：全量 50000，原生 32×32 + 增广
- 内层攻击：**PGD-7**, L_∞ , $\varepsilon = 8/255$
- ε 预热：前 5 轮从 0.5ε 线性升到 ε
- 损失： $(1 - \lambda) \text{CE}_{\text{clean}} + \lambda \text{CE}_{\text{adv}}$, $\lambda = 0.5$ （干净样本兜底**防塌缩**）
- 优化器：SGD ($\text{lr} = 0.1, \text{mom} = 0.9, \text{wd} = 5 \times 10^{-4}$)
- 调度：MultiStepLR 在 40 / 60 轮 $\div 10$ ，共 **80** 轮
- 梯度裁剪：上限 1.0

防“假鲁棒”要点

- 干净/对抗损失**各半**，避免模型塌缩（Clean/Robust 双低）。
- 评估用**强攻击**（APGD），并按强度阶梯 $\text{FGSM} < \text{PGD-20} < \text{APGD}$ 排查梯度混淆。
- Clean 与 Robust Acc **配对报告**。

防御效果：普通模型 vs 鲁棒模型

攻击强度阶梯：FGSM（弱）< PGD-20（标准基准）< APGD（最强）。同一模型 Robust Acc 应**依次递减**。

	Clean Acc	FGSM	PGD-20	APGD
普通模型	89.69%	1.88%	■	0.00%
鲁棒模型	■	■	■	■

读表方法： 对抗训练若真有效，鲁棒模型在三种攻击下的 Robust Acc 都应**明显高于**普通模型；代价是 Clean Acc 通常会下降（鲁棒性—准确率权衡）。

■ 处为最终 80 轮训练结果，待训练完成后填入。

本章内容

- 1 背景与实验目标
- 2 模型训练
- 3 DeepFool 攻击方法（核心）
- 4 攻击实验与结果
- 5 对抗训练（防御）
- 6 讨论与总结**

DeepFool 的优点与局限

优点

- 扰动**最小**、几乎不可见
- **无需手调**超参数
- 迭代少、**速度快**
- 几何直觉清晰、**易讲解**
- 提供可比的鲁棒性度量

局限

- 默认**无定向**（不能指定目标类）
- 扰动卡在边界、较**“脆”**（易被量化/压缩抵消，靠过冲缓解）
- 纯攻击**强度**略逊于 C&W
- **不适合**用于对抗训练

与其他攻击方法的关系

方法	思路	特点
FGSM	沿梯度符号走一步	单步、 L_∞ 、扰动大
PGD	在 ε 球内迭代最大化损失	迭代、 L_∞ 、训练/评估都用
APGD	自适应步长 + 重启	强评估基线 (AutoAttack 组件)
DeepFool	投影到最近决策边界	迭代、最小扰动、快、几何
C&W	优化最小 L_2 扰动	最强、扰动极小、较慢

- DeepFool 与 C&W 同属 “**最小扰动**” 路线（评估到边界的距离）。
- FGSM / PGD / APGD 属 “**固定预算**” 路线（关注给定 ε 内的破坏力）。

- ① **训练模型**: CIFAR 版 ResNet-18 (conv1 改 3×3 、去 maxpool) , 测试准确率 **88.53%**, 作为基线。
- ② **DeepFool 攻击**: 局部线性化、投影到最近决策边界, 求**最小** L_2 **扰动**; 实测平均 L_2 仅 **0.16**, 约 FGSM 的 $1/11$, 骗过率却相当。
- ③ **对抗训练**: 用 PGD (非 DeepFool) 做 min-max 训练得到鲁棒模型; 诚实评估需 Clean/Robust 配对 + 强攻击 (APGD) 。

一句话

攻击 (DeepFool) 量出模型有多脆, 防御 (PGD 对抗训练) 把它补强; 评估必须**诚实**, 否则 “假鲁棒” 一戳就破。

-  S.-M. Moosavi-Dezfooli, A. Fawzi, P. Frossard. DeepFool: a simple and accurate method to fool deep neural networks. CVPR 2016.
-  I. Goodfellow, J. Shlens, C. Szegedy. Explaining and Harnessing Adversarial Examples. ICLR 2015.
-  A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR 2018.
-  N. Carlini, D. Wagner. Towards Evaluating the Robustness of Neural Networks. IEEE S&P 2017.

谢谢！

欢迎提问与讨论